TRIP PLANNER PWA · FICHE MODULE · MEMBRE 2

Module 2 — Gestion des voyages

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
trips	**Synchronisation offline-first (service worker + IndexedDB)**	**Déploiement et pipeline CI**

1. Objectif du module

Gérer le voyage, objet central de l'application : informations générales, budget, devise de référence, préférences qui pilotent la checklist, participants et droits. Le module garantit aussi que l'application continue de fonctionner sans réseau : lecture complète des voyages ouverts et écritures mises en file d'attente puis rejouées à la reconnexion.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`title, destination`	texte	2 à 120 caractères
`country`	code ISO 3166	sert au calcul de la saison (hémisphère)
`startDate, endDate`	dates	fin ≥ début
`budget`	nombre	≥ 0 (0 = pas de suivi de budget)
`baseCurrency`	code ISO 4217	toutes les dépenses sont converties dans cette devise
`coverUrl`	texte | null	image de couverture
`preferences`	objet	tripType, activities[], accommodation, transport, travellers {adults, children, infants}, specialNeeds[]
`owner`	ref users	créateur du voyage
`members[]`	objets	{ user, role: owner | editor, joinedAt }
`shareToken`	texte	optionnel, unique (index sparse)
`createdAt, updatedAt`	dates	`updatedAt` sert au last-write-wins

3. Logique métier

Création

- Validation zod : titre, destination, pays, dates (fin ≥ début), budget ≥ 0, devise, préférences avec valeurs autorisées uniquement (enums).
- Le créateur devient **propriétaire** et premier membre (`role: owner`).
- Dans **la même transaction**, la checklist est générée à partir des préférences (appel au module 5).

- **Idempotence** : le client peut fournir l'`_id` (créé hors ligne). Si ce voyage existe déjà et que l'utilisateur en est membre, il est simplement renvoyé, ce qui évite les doublons quand la file est rejouée deux fois.

Lecture et permissions

- Un utilisateur ne voit que les voyages où il figure dans `members` ; la vérification se fait à **chaque appel** (`loadTripForMember`).
- Les modules enfants stockent la référence du voyage : une permission = une seule lecture du voyage (règle 3 du modèle).
- Tout membre peut lire et modifier le voyage et son contenu.
- Seul le **propriétaire** peut supprimer le voyage, inviter, retirer un participant et gérer le lien de partage.
- Un participant peut quitter le voyage ; le propriétaire ne peut pas (il doit le supprimer).
- Liste paginée avec recherche (titre, destination) et tri (date de début, création, titre).

Participants

- Invitation par email : le compte doit exister et être actif, sinon 404 avec un message explicite.
- Pas de doublon : inviter quelqu'un déjà présent renvoie 409.

Modification et suppression

- Chaque modification accepte `clientUpdatedAt` (heure réelle de la modification par l'utilisateur). Si le serveur a une version plus récente, il refuse avec **409** et renvoie sa version : **last-write-wins**.
- Si les préférences, les dates ou le pays changent, la checklist est **resynchronisée** automatiquement.
- Suppression en **cascade dans une transaction** : lieux, dépenses, checklist, journal, puis le voyage (opérations séquentielles, MongoDB n'autorisant pas les opérations parallèles dans une même session).
- `GET /trips/:id/full` renvoie le voyage et tout son contenu en une requête, pour en garder une copie hors ligne.

4. Fonctionnalité avancée — Synchronisation offline-first (service worker + IndexedDB)

Copie locale (IndexedDB, Dexie) : chaque voyage ouvert est téléchargé avec ses lieux, dépenses, checklist et journal. Les écrans lisent cette copie (requêtes « live »), donc ils fonctionnent sans réseau.

Écriture optimiste + file d'attente (outbox) : chaque création, modification ou suppression est appliquée immédiatement en local puis enregistrée dans la table `outbox` : entité, opération, voyage, document, contenu, heure de la modification.

Identifiants générés côté client (ObjectId) : un élément créé hors ligne a déjà son identifiant définitif, ce qui rend le rejeu idempotent et permet de chaîner « créer puis modifier ».

Rejeu dans l'ordre d'enregistrement :

– succès → l'op est retirée et la version serveur remplace la copie locale (sauf s'il reste d'autres ops sur ce document) ;

– erreur réseau, 5xx ou 429 → arrêt et nouvel essai plus tard ;

– 401 → l'indicateur affiche « Log in to sync » ;

– **409** → la version du serveur est gardée et un message prévient l'utilisateur ;

– 404 → l'élément est supprimé localement ;

– autre erreur 4xx → l'op est abandonnée.

Déclencheurs du rejeu : retour du réseau, retour sur l'onglet, test de connectivité toutes les 15 s, bouton de l'indicateur, et **Background Sync** : le service worker rejoue la même file même quand l'application est fermée

(Chrome/Edge).

Un **verrou** (Web Locks API) empêche la page et le service worker de rejouer la file en même temps.

Fusion au téléchargement : un document qui a encore des modifications en attente n'est jamais écrasé par la version serveur ; les documents supprimés sur le serveur sont retirés localement.

Service worker (`public/sw.js`) : stratégies de cache (voir tableau ci-dessous), page de secours `/offline`, nettoyage des anciens caches.

Indicateur de synchro visible en permanence : Synced, Offline · N pending, Syncing…, Log in to sync.

Installation : manifest (icônes 192/512 et maskable, mode standalone), bouton « Install » personnalisé ; sur iOS, guide « Ajouter à l'écran d'accueil ».

Stratégies de cache du service worker

Requête	Stratégie
Pages visitées	réseau d'abord, sinon copie en cache, sinon `/offline`
`/_next/static` , icônes	cache d'abord (fichiers versionnés)
Lectures API d'affichage (forum, taux, résumé budget, route…)	stale-while-revalidate
Posts du forum ouverts	réseau d'abord, 50 derniers conservés
Tuiles OpenStreetMap, images	cache d'abord (400 max)
Données des voyages	réseau uniquement : la copie hors ligne est dans IndexedDB

5. API REST (préfixe `/api/v1` , réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
`GET / POST`	`/trips`	membre	mes voyages / créer
`GET / PATCH / DELETE`	`/trips/:tripId`	membre (suppression : propriétaire)	détail, modification, suppression en cascade
`GET`	`/trips/:tripId/full`	membre	voyage complet pour le hors ligne
`POST`	`/trips/:tripId/cover`	membre	image de couverture
`POST`	`/trips/:tripId/members`	propriétaire	inviter par email
`DELETE`	`/trips/:tripId/members/:userId`	propriétaire ou soi-même	retirer / quitter

6. Écrans (front Next.js)

- Liste « My trips » (à venir / en cours / passés, badge « Not synced »)
- Création d'un voyage avec le formulaire de préférences
- En-tête du voyage et onglets ; vue d'ensemble (statistiques, jour par jour, participants)
- Paramètres : modifier, couverture, participants, lien de partage, suppression
- Bandeau hors ligne, indicateur de synchro, bouton d'installation, page `/offline`

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/trips/ (trip.model, trip.routes, trip.service)`
- `src/middleware/tripAccess.ts`
- `src/lib/sync.ts (contrôle last-write-wins)`
- `src/config/db.ts (transactions)`

Front — trip-planner-web

- `src/app/trips/page, trips/new, trips/[id]/layout, page, settings`
- `src/components/trips/TripForm.tsx, TripContext.tsx`
- `src/lib/db.ts, src/lib/sync.ts, src/lib/hooks.ts, src/lib/objectId.ts`
- `public/sw.js, src/app/manifest.ts, src/app/offline`
- `src/components/ServiceWorker.tsx, SyncIndicator.tsx, InstallPrompt.tsx`

8. Liens avec les autres modules

- **Modules 3, 4, 5, 6** : ils dépendent du voyage (référence et permissions) et utilisent la file hors ligne pour leurs écritures.
- **Module 5** : génération de la checklist à la création et à chaque changement de préférences.
- **Module 1** : suppression d'un compte = suppression de ses voyages ; lien de partage et export.
- **Module 7** : le service worker met en cache les posts du forum et affiche les notifications push.

9. À démontrer à la soutenance

- ☐ Créer un voyage : la checklist apparaît immédiatement.
- ☐ Inviter un autre compte, qui voit le voyage.
- ☐ DevTools → Network → Offline : recharger, tout reste lisible ; ajouter des éléments, puis montrer « Offline · N pending » et la table `outbox` dans IndexedDB.
- ☐ Revenir en ligne : synchronisation automatique.
- ☐ Conflit : modifier le même lieu sur deux navigateurs, montrer que la version la plus récente gagne.
- ☐ Installer l'application ; montrer le manifest et le service worker dans DevTools → Application.

10. Reste à faire

- Pipeline GitLab CI : lint, build (et tests) à chaque push.
- Déploiement HTTPS du front et de l'API, et sauvegarde quotidienne de la base.
- Vérifier le Background Sync avec l'application fermée.
- Audit Lighthouse (performance ≥ 80 sur mobile).

11. Questions probables du jury

Pourquoi last-write-wins et pas une fusion ?
Plus simple et prévisible ; c'est la mitigation prévue par le cahier des charges pour le risque « synchronisation trop longue à développer ».

Pourquoi ne pas mettre les données des voyages dans le cache du service worker ?
Une réponse ancienne écraserait des modifications locales plus récentes ; IndexedDB est la seule source hors ligne.

Que se passe-t-il si la même modification est envoyée deux fois ?
Les créations sont idempotentes (id client), les suppressions aussi, et une modification déjà appliquée est rejetée en 409 sans effet.